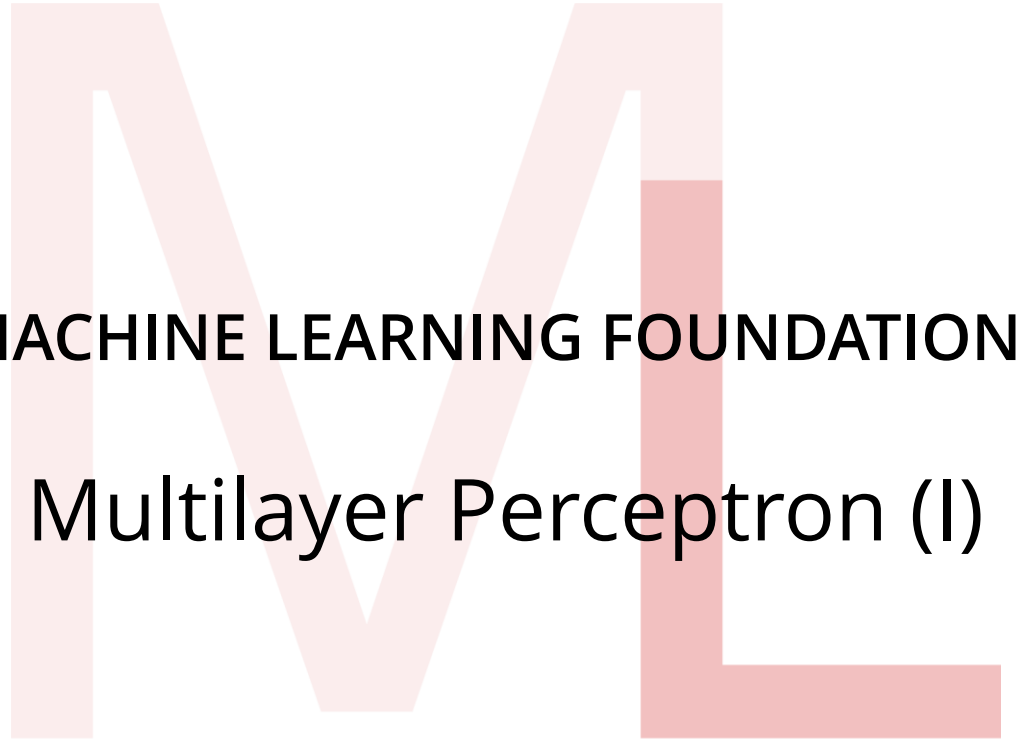
MACHINE LEARNING FOUNDATIONS

Multilayer Perceptron (I)



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Agenda

01	Recap
02	From Linear to Nonlinear models
03	Activation functions and softmax regression
04	Forward and backward propagation
05	Universal approximators

Recap

- Affine transformations are linear mappings with added bias

$$\hat{y} = w_1 x_1 + \cdots + w_d x_d + b$$

or assuming $\mathbf{x}, \mathbf{w} \in \mathbb{R}^d$

$$\hat{y} = \boldsymbol{w}^\top \mathbf{x} + b$$

Recap

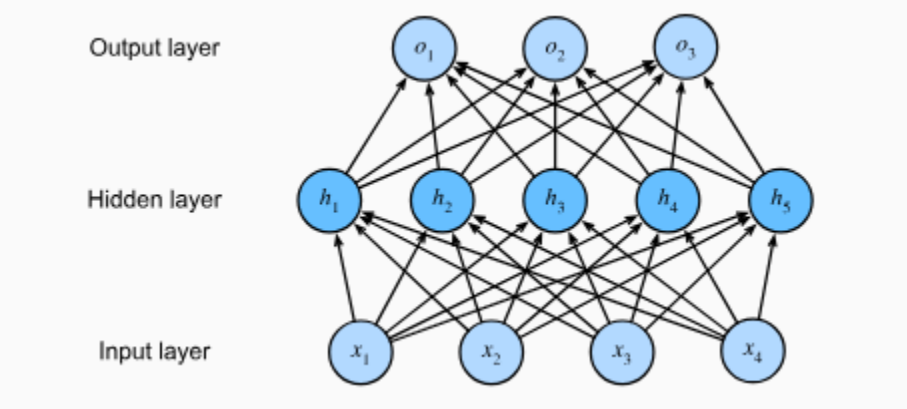
- This model assumes labels relate linearly to input data.
  - Linearity is a strong assumption that may not always hold.
  - More complex models may be needed for **non-linear relationships**.
- Linearity assumes **monotonic relationships** between features and outcomes.
  - Monotonicity means increasing a feature always increases or decreases output.
- Real-world relationships, like income and loan repayment, often violate strict linearity.
  - If your income goes from 0 to 50k, the increase in likelihood of payment will increase a lot
  - If your income goes from 1M to 1.05M, the increase is no so big

Limitations of linear models

- Some features have non-monotonic effects, such as body temperature's effect on health.
  - Both very low and very high temperatures can indicate health risks, violating monotonicity.
- Or even more clear, the relation between pixel brightness of an image and the likelihood that the image depicts a dog (versus a cat)
  - Linear models assume that the prediction cat/dog comes directly from pixel brightness

From linear to nonlinear

- To overcome the limitations of linera models, we need to incorporate one (or more) **hidden layers**



- This is what we call **Multilayer Perceptron**. This MLP has
  - Input layer (does not involve any calculation)
  - Hiddel layer
  - Output layer
- We say it is a 2-layer or **1-hidden-layer** MLP

Hidden layer

- Formally, we denote by $\boldsymbol{X} \in \mathbb{R}^{nxd}$ a batch of n examples, each example has d features (inputs).
- For a one-hidden-layer MLP with h hidden units (neurons) and one single unit in the output layer, we have
- Set of paremeters:
  - $W^{(1)} \in \mathbb{R}^{dxh}, b^{(1)} \in \mathbb{R}^{1xh}$
  - $W^{(2)} \in \mathbb{R}^{hx1}, b^{(2)} \in \mathbb{R}^{1x1}$

Hidden layer

- Calculations

$$H = XW^{(1)} + b^{(1)}$$
$$O = HW^{(2)} + b^{(2)}$$

- The hidden layer transforms X into $H \in \mathbb{R}^{nxh}$
- The output layer transforms H into $O \in \mathbb{R}^{nx1}$

- **But**, if we denote $W = W^{(1)}W^{(2)}, b = b^{(1)}W^{(2)} + b^{(2)}$

$$O = \left(XW^{(1)} + b^{(1)}\right)W^{(2)} + b^{(2)} = XW^{(1)}W^{(2)} + b^{(1)}W^{(2)} + b^{(2)} = XW + b$$

The model is **still linear**!!

Hidden layer

- In order to obtain the potential of MLP, we need to add a nonlinearity: the **activaction function**
- In a general way, we denote an activation function as $\phi: \mathbb{R} \rightarrow \mathbb{R}$. The activation function will be applied to each hidden unit

$$H = \phi\left(XW^{(1)} + b^{(1)}\right)$$
$$O = HW^{(2)} + b^{(2)}$$

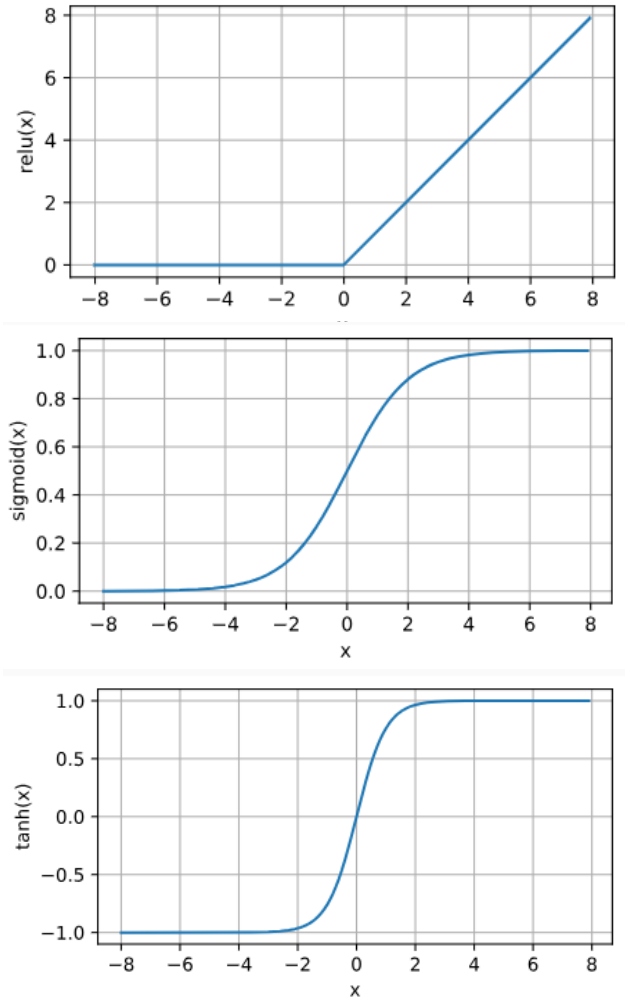
And now, if we want to express O in terms of X, we have

$$O = HW^{(2)} + b^{(2)} = \phi\left(XW^{(1)} + b^{(1)}\right)W^{(2)} + b^{(2)}$$

And the relation is nonlinear.

Activation functions

- Every activation function is applied after the affine transformation on each hidden unit
- Most popular activation functions
  - Rectified linear unit (ReLU).
    - $ReLU(x) = \max(x, 0)$
  - Sigmoid function
    - $sigmoid(x) = \frac{1}{1+\exp(-x)}$
  - Hyperbolic tangent (tanh)
    - $\tanh(x) = \frac{1-\exp(-2x)}{1+epx(-2x)}$



Output function

- As we have seen, the MLP in the example had an output layer with one unit, so the ouput of the MLP is $O \in \mathbb{R}^{nx1}$
- Coming from the logistic regression, one can think on the ouput layer to have also an activation function $\phi = \sigma$ (sigmoid)
  - If $o \leq 0.5$ we asume x belongs to class 0
  - If $o > 0.5$ we asume x belongs to class 1
    - The output can be seen as the probability of beloning to class 1
- Does it mean we can only handle 2-class problems?

Softmax regression

- MLPs can obviously handle n class
- Suppose a problem where an image has to be classified as picture of a “dog”, “cat” or “chicken”
  - The natural impulse is to represent the class of each sample as $y \in \{0, 1, 2\}$
- This is a great way of storing information when the categories have some natural ordering among them
  - Think on predicting $\{baby, toddler, adolescent, young, adult, geriatric\}$

Softmax regression

- In general, classfication problems do not come with natural ordering among the classes
- To represent this kind of categorical data we use the **one-hot encoding**
  - Vector with as many components as categories
  - The component corresponding to a particular instance is set to 1 (and the rest to 0's)
- In our example, "dog" will be (1,0,0), "cat" will be (0,1,0) and "chicken" will be (0,0,1)
  - $y \in \{(1,0,0), (0,1,0), (0,0,1)\}$

Softmax regression

- Since the classes do not come with natural ordering, we can't handle a multiclass problema with a single ouput
  - Again, if $y \in \{0,1,\ldots,9\}$, we can think on having a single unit in the output layer and performs some kind of regression (in fact, this is called ordinal regression)
- Now, we need to estimate the conditional probabilities associated with all posible classes, and therefore we need a model with **multiple outputs**
  - We need as many outputs as classes
  - We need all the probabilities to sum up 1
    - Actually, we could miss the last output, but for the sake of simplicity we set number of outputs = number of classes
  - In fact, 2-class problemas are usually treated with two outputs

Softmax regression

- For a one-hidden-layer MLP with h hidden units (neurons) and q units in the output layer, we have
- Set of paremeters:
  - $W^{(1)} \in \mathbb{R}^{dxh}, b^{(1)} \in \mathbb{R}^{1xh}$
  - $W^{(2)} \in \mathbb{R}^{hxq}, b^{(2)} \in \mathbb{R}^{1xq}$
- Calculations

$$H = \phi\left(XW^{(1)} + b^{(1)}\right)$$
$$O = HW^{(2)} + b^{(2)}$$

- The hidden layer transforms X into $H \in \mathbb{R}^{nxh}$
- The output layer transforms H into $O \in \mathbb{R}^{nxq}$

Softmax regression

- But… there is no guarantee that $O \in \mathbb{R}^{nxq}$ sums up to one row-wise (conditional probabilities)
  - In fact, there is no guarantee that the outputs are even nonnegative
- The output layer can be equipped with a **softmax** function, that is applied to each unit of the output layer in the following way

$$\hat{y} = softmax(o), \qquad where\ \hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

- Properties of softmax function
  - Every output is nonnegative, due to exp function
  - The set of outputs sum 1 due to normalization
  - The softmax operatiron preserves the ordering
    - The largest coordinate of o correspondons to the largest probability of y

Softmax regression

- From now on we will design our MLPs with
  - As many outputs as classes
  - Assuming that the value of each output will be transformed into a probability of the belonging to the corresponding class
- Note: in practice, we do not equip the MLP with the softmax activation function. In contrast, we let the ouput to be any value and it is the loss function the respondible of performing the softmax operation

Forward and backward propagation

- The automatic calculation of gradientes simplifies the implementation of deep learning algorithms
- The calculations of a neural netowork are done in two steps: **forward propagation and backward propagation** (or backpropagation)

Forward

- For the sake of simplicity, assume we have $X \in \mathbb{R}^{nxd}$ and that our hidden layer does not include a bias term
- The calculations of the hidden layer are

$$Z = W^{(1)}X$$
$$H = \phi(Z)$$

- The calculations of the outpur layer are

$$O = W^{(2)}H$$

- Now, assuming that the loss function is l we can calculate the loss for each example as

$$L = \sum_i l(o_i, y_i)$$

- Since still we have not seen regularization, we assume

$$J = L$$

Backpropagation

- The concept of backpropagation referes to the method of calculating the grandient of the parameters of a neural network.
- Basically, the method traverses the network in reverse order applying the **chain rule** from calculus
- Recall that the objective of backpropagation is to obtain $\frac{\partial J}{\partial W^{(1)}}, \frac{\partial J}{\partial W^{(2)}}$

Backpropagation

- $\frac{\partial J}{\partial L} = 1$ since $J = L$
- $\frac{\partial J}{\partial O} = \frac{\partial J}{\partial L}\frac{\partial l}{\partial O} = \frac{\partial l}{\partial O} \in \mathbb{R}^{nxq}$ since
- $\frac{\partial J}{\partial W^{(2)}} = \frac{\partial J}{\partial O}\frac{\partial o}{\partial W^{(2)}} = H^{\top}\frac{\partial J}{\partial o} \in \mathbb{R}^{hxq}$-> we can now update $W^{(2)}$
- $\frac{\partial J}{\partial H} = \frac{\partial J}{\partial O}\frac{\partial O}{\partial H} = \frac{\partial J}{\partial O}\mathrm{W}^{(2)^{\top}} \in \mathbb{R}^{nxh}$
- $\frac{\partial J}{\partial Z} = \frac{\partial J}{\partial H}\frac{\partial H}{\partial Z} = \frac{\partial J}{\partial H} \odot \phi'(Z)$
- $\frac{\partial J}{\partial W^{(1)}} = \frac{\partial J}{\partial Z}\frac{\partial Z}{\partial W^{(1)}} = X^{\top}\frac{\partial J}{\partial Z} \in \mathbb{R}^{dxh}$-> we can now update $W^{(1)}$

Backpropagation

- When training neural networks, forward and backward depend on each other
  - We alternate forward propagation with backpropagation
- Note that backpropagation reuses the stored intermediate values from forward propagation
  - We need to retain intermediate values until backpropagation is complete
- Memory allocation
  - Training NN requires more memory that plain prediction
  - The size increases with the size of the NN and the input size

Universal Approximators

- It is well known (papers from 1980s) that one-hidden-layer neural networks can model any function
  - Given enough neurons (possibly absurdly many)
  - And the right set of weights (maybe impossible to learn using gradient descent)
  - Can model any function with a specific precision ϵ
- We say that one-hidden-layer are **universal approximators**
  - In terms of regression, they can model any function
  - In terms of classification, they can model any decisión boundary

Universal Approximators

- Mathematically, this fact can be proven with some conditions
- Intuitively, we can think in the following way:
  - A neuron equipped with relu can model a picewise linear function being zero until any point $a \in \mathbb{R}$ and then linear with arbitrary slope $b \in \mathbb{R}$
  - When adding two relus, we can model a “step”
  - We can adjust this step to “cover” a narrow Interval of the function
  - By adding aribtrary many “steps”, we can adjust any function

Universal Approximators

- If one-hidden-layers are universal approximators, why “going deeper”
  - The universal approximator theorem proves the existence of this neural network but does not stablish
    - The number of neurons needed
    - If the set of weights is reachable via optimization
  - And…what about generalization?
- We will see that **deeper neural networks** can also model complex functions
  - With a reduced number of neurons
  - By adding more nonlinearities to the model

Breaking the simmetry

- Although we have not mentioned before, the way we initialize the neurons of a hidden layer is important
- In our codes, bias have been initialized to zero, while weights to random numbers
- If we initialize all the neurons of a hidden layer to the same constant value
  - When updating, we will get the same derivative
  - All the neurons will learn the same function
  - The one-hidden-layer MLP will work as a Logistic Regression!